# Group Project

## Phase 1

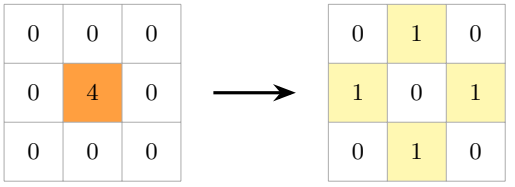
DM857 Introduction to Programming
DS830 Introduction to Programming

## 1 Introduction

In this project, you will implement and explore a simple model of avalanches in a pile of sand. The model is called the *Abelian sandpile model*. Although its rules are local and short, repeated applications of the rules can lead to large avalanches and interesting patterns.

The sandpile is represented by a rectangular grid. Each cell of the grid contains a non-negative integer: the number of grains of sand at that position. A grain can be added to any cell. If a cell contains at least four grains, it is unstable and must topple. When a cell topples, it loses four grains and each of its four orthogonal neighbours receives one grain. A grain that would move outside the grid is lost.

For example, if the central cell of a $3 \times 3$ grid contains four grains, one toppling produces:




before toppling after one toppling

The central cell may have more than four grains. A cell with height 9, for example, must topple twice before it can become stable. A toppling may make a neighbour unstable, which can in turn cause an avalanche.

Your task is to develop the code needed to initialise and manage a simulation of this system and to analyse its evolution. You are provided with a visualisation library to help you display the sandpile as the simulation runs. Your group will also implement a command-line interface to control the simulation.

## 2 The Sandpile Model

### 2.1 Grid representation

The complete pile must be represented by an ordinary two-dimensional Python list of integers. Do not use NumPy arrays, classes, custom containers, or global mutable state for the model.

```
pile = [
    [0, 1, 0],
    [2, 3, 1],
]
```

The value `pile[row][column]` is the height of that cell. Rows are numbered from top to bottom, beginning with zero. Columns are numbered from left to right, also beginning with zero. Every height must be a non-negative integer.

The program must maintain three ordinary integer counters:

- `grains_added`: total number of grains added during the simulation;
- `grains_lost`: total number of grains that left through an edge;
- `topplings`: total number of topplings performed.

## 2.2 Toppling rule

A cell is unstable if its height is at least four. For every toppling at position $(r, c)$:

1. subtract four from `pile[r][c]`;
2. add one grain to each valid north, south, east, and west neighbour;
3. count every grain directed outside the grid as a lost grain.

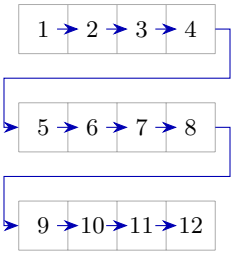
There are no diagonal neighbours. One grain is lost for each of the four directions that points outside the grid. For example, on a grid with at least two rows and two columns, a corner toppling loses two grains. On a $1 \times 1$ grid, all four grains are lost.

After adding one grain, the pile must be relaxed: keep toppling unstable cells until all cells have height at most three. The final stable pile is independent of the order in which valid topplings are processed.

## 2.3 Lexicographic scan with local backtracking

Use one deterministic scan order: visit columns from west to east within a row, and visit rows from north to south. Thus the scan begins at the top-left cell and normally proceeds lexicographically:

$$(0,0), (0,1), \ldots, (0, C-1), (1,0), \ldots, (R-1, C-1).$$




Example scan on a grid with three rows and four columns. The numbers show the order in which the cells are visited.

When a cell $(r, c)$ topples, its east and south neighbours occur later in this order and will be visited naturally. Its north and west neighbours occur earlier in the order and may already have been checked. The scan must therefore move backwards locally after a toppling. Figure 1 shows the four affected neighbours.

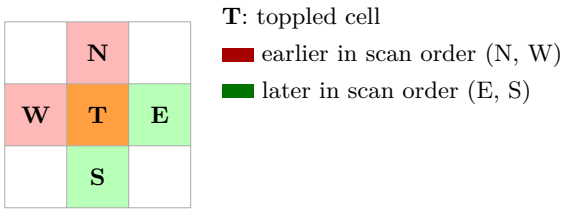



Figure 1: A toppling affects four orthogonal neighbours. North and west occur earlier in the lexicographic order and may need another visit; east and south occur later and will be reached naturally.

More precisely, immediately after toppling $(r, c)$, choose the next scan position as follows:

1. if $r > 0$, continue from $(r-1, c)$, the north neighbour;
2. otherwise, if $c > 0$, continue from $(r, c-1)$, the west neighbour;
3. otherwise, continue from $(0, 0)$.

Then continue normally in lexicographic order. Starting at $(r-1, c)$ is sufficient: after completing that row, the normal scan reaches row $r$ at column zero and therefore also revisits the west neighbour when needed.

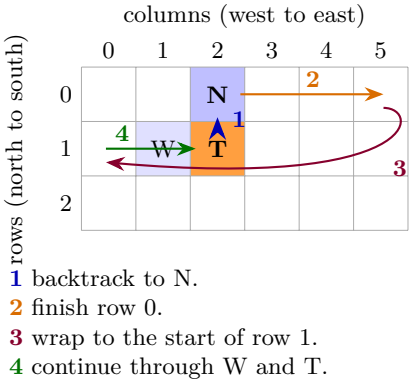



Figure 2: Local backtracking on a grid with six columns and three rows. T is at $(1, 2)$ and N is at $(0, 2)$. Column indices increase from left to right and row indices from top to bottom. The numbered arrows show the order followed by the scan after T topples.

Figure 2 illustrates this strategy on an example grid with six columns and three rows. The toppled cell is at row 1, column 2; the general rule above applies to grids of any positive dimensions.

Implement this algorithm with `while` loops whose row and column indices you control explicitly; nested `for` loops are not suitable because the scan position must sometimes move backwards. The scan may revisit cells many times during a large avalanche. It finishes when it advances beyond the bottom-right cell without needing another toppling.

# 3 Required Functions

Create a module called `sandpile.py`. It must define the following type alias[1] and five functions. Every function must have complete type hints, a docstring, and at least one doctest.

```
Pile = list[list[int]]

def create_pile(rows: int, columns: int) -> Pile:
    ...

def add_grain(pile: Pile, row: int, column: int) -> None:
    ...

def topple(pile: Pile, row: int, column: int) -> int:
    ...

def relax(pile: Pile) -> tuple[int, int]:
    ...

def is_stable(pile: Pile) -> bool:
    ...
```

### `create_pile`

```
create_pile(rows, columns) -> Pile
```

Return a rectangular grid with the requested number of rows and columns. Every cell must initially contain zero. The CLI must reject non-positive dimensions before calling this function.

[1] If you don't know/remember what a type alias is in Python, see the documentation: `https://docs.python.org/3/library/typing.html#type-aliases`.

### add_grain

```
add_grain(pile, row, column) -> None
```

Add exactly one grain to the selected cell. The function changes the supplied pile and returns `None`. The CLI is responsible for checking that the row and column are inside the grid.

### topple

```
topple(pile, row, column) -> int
```

Topple one unstable cell once. The function may assume that the selected cell has height at least four. It changes the pile and returns the number of grains lost through the boundary during this toppling.

### relax

```
relax(pile) -> tuple[int, int]
```

Repeatedly find and topple unstable cells until the pile is stable. Return:

```
(number_of_topplings, number_of_lost_grains)
```

The function should not print output.

### is_stable

```
is_stable(pile) -> bool
```

Return `True` precisely when every cell has height at most three, and `False` otherwise.

## 4 Command-Line Interface and Visualisation

Create a second module called `sandpile_cli.py`. It must contain a `main()` function that runs the terminal interface. At startup, ask the user for the number of rows and columns. Every simulation cycle then adds one grain at a randomly selected valid cell. The commands below are required:

| | |
|---|---|
| `run N` | Run $N$ random-addition cycles, then redraw the pile. |
| `history` | Display the current history plots without ending the program. |
| `reset` | Empty the pile and clear the counters and histories. |
| `help` | Print a short explanation of the available commands. |
| `quit` | Close the visualisation and end the program. |

For every cycle in `run N`, select randomly a `row` and `column` using the standard library module `random`, call the functions `add_grain`, and call `relax`.

Invalid commands, a missing or invalid value of $N$, and non-positive values of $N$ must be handled politely. The program must print a short explanation and continue running rather than terminating with an unhandled exception.

The supplied view is imported as follows:

```
from sandpile_support.view import SandpileView
```

Your CLI should create one `SandpileView` after the grid dimensions are known. After the requested cycles in `run N`, call:

```
view.update(pile, grains_added, grains_lost, topplings)
```

The three counters are maintained by your code and used by the view only as labels.

Reset keeps the grid dimensions and redraws the empty pile with zero counters. Consult the viewer module's docstring for usage instructions `help`.

Keep a clear modular structure, with each module responsible for a distinct part of the program. For example, the model module should implement the sandpile rules, not perform drawing; visualisation should be handled separately.

Do not modify the files in `sandpile_support`. The supplied code handles only drawing; it does not implement the sandpile rules or the terminal interface.

# 5 Data Collection and History Visualisation

The command-line interface is for controlling the simulation, not for reporting numerical results. Apart from help text and short messages about invalid input, do not print statistics, tables, or detailed summaries in the terminal.

During the simulation, your program must retain enough information to describe the history of each complete random-addition cycle. A cycle starts when one grain is added at a randomly selected cell and ends when the resulting avalanche has completely stopped. At minimum, retain the number of topplings and the number of grains that fall beyond the boundary for every cycle.

The precise data structures, variable names, helper functions, and moments at which data are stored are your design choices. Choose a representation that is clear, correct, and convenient for later plotting.

The `history` command must use Matplotlib to display a useful visual representation of the recorded histories. The visualisation must make it possible to inspect how avalanche activity and boundary losses evolve during a run. Decide for yourselves whether to use one figure or several figures, which plot types are most appropriate, and how to organise the axes and legends. Every plot must nevertheless be readable, have informative labels, and identify the displayed quantities clearly.

# 6 Coding Rules

Use only the Python standard library in `sandpile.py`. In `sandpile_cli.py`, Matplotlib may be used only for the supplied visualisation support and the history plots. The support package documentation explains how to install it.

Every module needs a module docstring. Every function needs type hints, a docstring, and at least one doctest. Do not use advanced language features that have not been covered in the course.

You must be able to run:

```
python -m doctest -v sandpile.py
python -m doctest -v sandpile_cli.py
```

without failing examples.

# 7 Hand-in

Submit one zip file named after your group, for example `group_B20.zip`. It must contain:

- a folder named `code` containing `sandpile.py`, `sandpile_cli.py`, and the unchanged supplied support package;
- a PDF report named `named_report.pdf`.

The report must be written in English, printable in black and white, and no longer than 8 pages. State the group name and the full names and SDU e-mail addresses of all group members.

Describe the program structure, the representation of the pile and histories, the toppling and relaxation algorithms, and the tests used to check correctness. Explain the history visualisation and briefly discuss an observation from a simulation run together with its history visualization.

Refer to the scoring matrix in the course description for the criteria used to assess your work.